Feature Selection in Statistical Machine Learning

Garvesh Raskutti

Lecture 7

Recap and outline

- We previously covered embedded methods based on regularized regression:

$$\widehat{\beta}_j = 0 \Leftrightarrow X_j \notin \widehat{S}$$

- Re-visiting identifiability and Lasso
- Today we cover MDI (mean decrease in impurity) which applies to decision tree ensembles (Random forests, gradient boosted decision trees)
- Quick intro to decision trees and ensembles
- Introducing MDI
- Examples

Recall identifiability challenge for linear regression (and generalized linear models)

Linear regression satisfies normal equations:

$$X^T X\widehat{\beta} = X^T Y$$

- If $X^T X$ not invertible (rank$(X) < p$), $\widehat{\beta}$ is ill-defined
- Occurs when we have extremely high correlation and/or $p > n$
- Feature selection is not identifiable (even if everything is linear)
- So to be clear there is no single *correct* way to do feature selection
- Many reasonable choices

Highly correlated features example

Consider a linear model where:

$$Y = X_1 + \frac{1}{2}X_2$$

but the features are perfectly correlated:

$$X_2 = 2X_1$$

If you fit a linear model

$$Y = \beta_1 X_1 + \beta_2 X_2,$$

any solution (β_1, β_2) of the form

$$\beta_1 + 2\beta_2 = 2$$

works.

High-dimensional $p > n$ example

Need to fit a linear model where $p > n$. For our data assume $Y^{(i)} = r_j$ and

$$\begin{aligned} X_j^{(i)} &= 1 \text{ if } i = j \\ X_j^{(i)} &= 0 \text{ if } i \neq j \end{aligned}$$

Each new sample selects a different mutation.
Then matrix $X^T X$ is a diagonal matrix with n diagonal entries 1 and the other $p - n$ entries are 0.
For $1 \leq j \leq n$, we get the equations:

$$\widehat{\beta}_j = r_j$$

while for $n + 1 \leq j \leq p$ we get the equations:

$$0.\widehat{\beta}_j = 0$$

which has infinite solutions.

Lasso resolves non-identifiability by finding *sparsest* solution

- $\|\beta\|_1$ encourages sparse solution due to point of non-differentiability (can be re-expressed as linear program where solution is at a vertex)
- For first example Lasso leads to $(\widehat{\beta}_1, \widehat{\beta}_2) = (0, 1)$
- For second example $\widehat{\beta}_j = r_j$ for $1 \leq j \leq n$, $\widehat{\beta}_j = 0$ for $n + 1 \leq j \leq p$
- Ridge regression or other approaches resolves non-identifiability in a different way
- So Lasso is not providing *correct* solution, just sparsest

Why are sparse solutions (less features selected) often preferred?

- Interpretability: Less features easier to interpret
- Default position: A feature is not significant unless there is strong empirical evidence to include it
- Occam's razor: Simplest solution is always the best
- Note the for prediction performance, selecting more features usually better (ridge regression often optimal in terms of mean-squared error)
- When using Lasso make sure output is interpreted correctly

Choosing regularization parameter λ

Recall Lasso objective function:

$$\widehat{\beta} \in \arg\min_{\beta} \sum_{i=1}^{n} (Y^{(i)} - \sum_{j=1}^{p} X_j^{(i)} \beta_j)^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

In matrix-vector form:

$$\widehat{\beta} \in \arg\min_{\beta} ||Y - X\beta||_2^2 + \lambda ||\beta||_1$$

- Each choice of λ gives very different solution
- Common problem in statistics/machine learning is choosing tuning parameters/hyper-parameters (e.g ridge regression, hyper-parameters in Bayesian statistics, e.t.c.)
- In simple terms λ trades off *sparsity* and *data fit*
- Larger λ means sparser solution, smaller λ means less sparse but better data fit

Standard principles and practices for choosing λ

- Choose λ to optimize hold-out prediction performance (often default) using cross-validation (often leads to non-sparse solutions)
- Tune λ to determine k best features
- Choose λ according to a different performance criteria (e.g. stability selection)
- Statistical theory suggests $\lambda \sim \sqrt{n \log p}$
- Again, no single correct solution and this is another important design choice

Housing data example

- Larger version of original housing dataset (from Kaggle)
- $n \approx 20,000$, $p = 8$
- Y - median house income

longitude	latitude	housing_me	total_rooms	total_bedroc	population	households	median_inco	median_house_value
-122.23	37.88	41	880	129	322	126	8.3252	452600
-122.22	37.86	21	7099	1106	2401	1138	8.3014	358500
-122.24	37.85	52	1467	190	496	177	7.2574	352100
-122.25	37.85	52	1274	235	558	219	5.6431	341300
-122.25	37.85	52	1627	280	565	259	3.8462	342200
-122.25	37.85	52	919	213	413	193	4.0368	269700
-122.25	37.84	52	2535	489	1094	514	3.6591	299200
-122.25	37.84	52	3104	687	1157	647	3.12	241400
-122.26	37.84	42	2555	665	1206	595	2.0804	226700
-122.25	37.84	52	3549	707	1551	714	3.6912	261100
-122.26	37.85	52	2202	434	910	402	3.2031	281500
-122.26	37.85	52	3503	752	1504	734	3.2705	241800
-122.26	37.85	52	2491	474	1098	468	3.075	213500
-122.26	37.84	52	696	191	345	174	2.6736	191300
-122.26	37.85	52	2643	626	1212	620	1.9167	159200
-122.26	37.85	50	1120	283	697	264	2.125	140000
-122.27	37.85	52	1966	347	793	331	2.775	152500
-122.27	37.85	52	1228	293	648	303	2.1202	155500
-122.26	37.84	50	2239	455	990	419	1.9911	158700
-122.27	37.84	52	1503	298	690	275	2.6033	162900
-122.27	37.85	40	751	184	409	166	1.3578	147500

Housing data- λ selected by cross-validation

```
df = pd.read_csv("housing.csv")
df = df.dropna().reset_index(drop=True)
X = df.drop(columns="median_house_value" )
y = df["median_house_value"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lasso = LassoCV(cv=5, random_state=0)
lasso.fit(X_train_scaled, y_train)

lasso_coef = pd.Series(lasso.coef_, index=X.columns)
lasso_selected = lasso_coef[lasso_coef != 0].index.tolist()

lasso_selected
```

```
['longitude',
 'latitude',
 'housing_median_age',
 'total_rooms',
 'total_bedrooms',
 'population',
 'households',
 'median_income']
```

Housing data- $\lambda = 1000$

```
df = pd.read_csv("housing.csv")
df = df.dropna().reset_index(drop=True)
X = df.drop(columns="median_house_value" )
y = df["median_house_value"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lasso = Lasso(alpha=1000)
lasso.fit(X_train_scaled, y_train)

lasso_coef = pd.Series(lasso.coef_, index=X.columns)
lasso_selected = lasso_coef[lasso_coef != 0].index.tolist()

lasso_selected
```

```
: ['longitude',
   'latitude',
   'housing_median_age',
   'total_bedrooms',
   'population',
   'households',
   'median_income']
```

Housing data- $\lambda = 5000$

```
df = pd.read_csv("housing.csv")
df = df.dropna().reset_index(drop=True)
X = df.drop(columns="median_house_value")
y = df["median_house_value"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lasso = Lasso(alpha=5000)
lasso.fit(X_train_scaled, y_train)

lasso_coef = pd.Series(lasso.coef_, index=X.columns)
lasso_selected = lasso_coef[lasso_coef != 0].index.tolist()

lasso_selected
```

```
['longitude',
 'latitude',
 'housing_median_age',
 'total_bedrooms',
 'median_income']
```

Housing data- $\lambda = 10000$

```
df = pd.read_csv("housing.csv")
df = df.dropna().reset_index(drop=True)
X = df.drop(columns="median_house_value")
y = df["median_house_value"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lasso = Lasso(alpha=10000)
lasso.fit(X_train_scaled, y_train)

lasso_coef = pd.Series(lasso.coef_, index=X.columns)
lasso_selected = lasso_coef[lasso_coef != 0].index.tolist()

lasso_selected
```

```
['latitude', 'housing_median_age', 'median_income']
```

Feature selection methods for decision tree ensembles

- Lasso/generalized linear models well-suited to linear relationships
- Many different kinds of interactions can go on
- Decision trees and decision tree ensembles (random forests and gradient boosted trees) often useful in these cases (especially tabular data)

When are they most useful

- If data is in tabular form
- Originally for when features and responses are categorical with small number of categories
- A lot of work on regression trees and extending to continuous responses and features
- Less suited to image and language data where data doesn't fit as naturally in a table

Embedded feature selection in decision tree ensembles

- MDI (mean decrease in impurity)
- How much does each feature decrease impurity (makes split more pure)
- Embedded since MDI is used to compute each decision tree in ensemble
- We take an average over all the trees

Toy example - Titanic

Gender	Class	Survived
Female	1	1
Female	2	0
Female	1	1
Female	1	1
Female	2	1
Male	1	0
Male	1	0
Male	2	0
Male	2	0
Male	2	0

Measuring information gain/impurity for each decision

K categories
Gini (fast and easy):

$$1 - \sum_{k=1}^{K} p_k^2$$

Entropy (slower but connecte to information theory):

$$-\sum_{k=1}^{K} p_k \log p_k$$

For Y (deceased vs survived)
Gini: $1 - 0.4^2 - 0.6^2 = 0.48$
Entropy: $-0.4 \log_2(0.4) - 0.6 \log_2(0.6) = 0.971$

Decision tree computation

- How do we split the root node into branches (use Gini)?
- Root node impurity 0.48 from previous slide
- We could split on either Gender or Class
- The decision tree algorithm chooses which we split on based on biggest drop in MDI
- If we split on Class, for Class 1 3 survived and 2 died, so probability is 0.6 of survival
  Gini: $1 - 0.4^2 - 0.6^2 = 0.48$
- For Class 2, 1 survived and 4 died Gini: $1 - 0.2^2 - 0.8^2 = 0.32$
- Weighted impurity is:

$$\frac{5}{10} \times 0.48 + \frac{5}{10} \times 0.32 = 0.40$$

- Since original impurity was 0.48, the MDI is $0.48 - 0.40 = 0.08$

Decision tree computation cont.

- Now let's see what happens if we split on Gender
- For Female, 4 survived and 1 died Gini: $1 - 0.2^2 - 0.8^2 = 0.32$
- For Male, all 5 died (pure) Gini: $1 - 0^2 - 1^2 = 0$
- Weighted impurity is:

$$\frac{5}{10} \times 0.32 + \frac{5}{10} \times 0 = 0.16$$

- Since original impurity was 0.48, the MDI is $0.48 - 0.16 = 0.32$
- Since Gender has a much higher MDI than Class, we first split along Gender

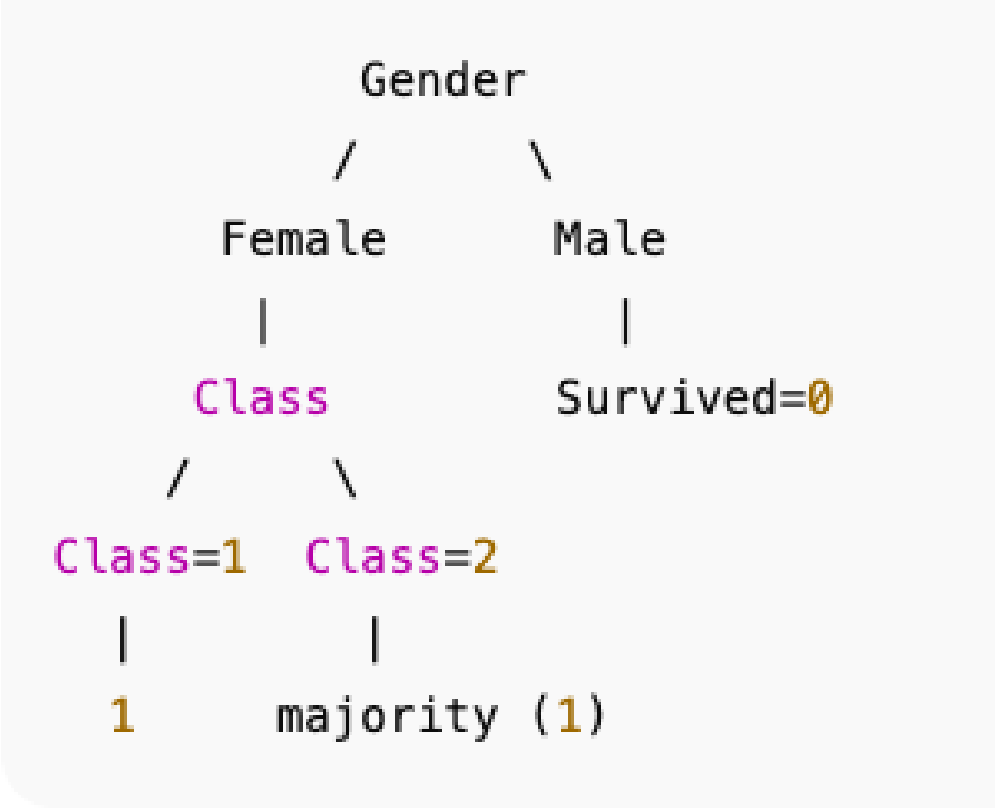
Decision tree computation cont.

- Now we have the first split and we want to determine how to split further
- Look at each leaf node. For Male all 5 died so Gini is 0 and we stop (pure node)
- For Female, Gini is 0.32, if we split along Class 1, 3 survived and 0 died (pure), so Gini is 0
- For Class 2, 1 survived and 1 died so Gini is $1 - 0.5^2 - 0.5^2 = 0.5$
- Weighted impurity:

$$\frac{3}{5}0 + \frac{2}{5} \times 0.5 = 0.20$$

- So MDI is $0.32 - 0.20 = 0.12$

Final decision tree and feature selection/ranking



Ranking features by MDI:

Gender MDI: 0.32

Class MDI (need to weight based on Gender): $5/10 \times 0.16 = 0.08$

Limitations of simple decision trees

- For this simple example a single decision tree works fine
- Decision trees are nice and interpretable
- When sample size, number of categories/continuous features and number of features grow fitting a single tree is very problematic
- Each feature is selected locally and greedily
- Leads to great instability/high variance
- If your tree is deep, likely to completely overfit the data
- If the tree is shallow, likely to lead to a high bias/not fit data well
- A single tree is rarely a rich enough model class to describe the whole dataset

Solution: Ensembles of trees

- Idea to address this is ensemble learning
- Ensemble learning: consider $\widehat{f_1}$, $\widehat{f_2}$,..., $\widehat{f_M}$ a set of M *weak learners* like decision trees
- Can we combine weak learners/decision trees

$$\widehat{f} = \sum_{m=1}^{M} \alpha_m \widehat{f_m}$$

- So that $\widehat{f}$ has good prediction performance?
- Two broad ideas:
  - Bagging (Random Forests): Take high-variance trees and use averaging to reduce the variance
  - Boosting (Gradient Boosted Trees): Take individual trees with high bias and systematically keep reducing the bias

Random forests: Bagging (boosted aggregation)

- Consider $\widehat{f_1}$, $\widehat{f_2}$,..., $\widehat{f_M}$ a set of M *weak learners* like decision trees
- Idea behind bootstrap aggregation: If $\widehat{f_1}$, $\widehat{f_2}$,..., $\widehat{f_M}$ are independent and $\widehat{f} = \frac{1}{M}\sum_{m=1}^{M}\widehat{f_m}$,

$$\text{Var}(\widehat{f}) = \text{Var}(\frac{1}{M}\sum_{m=1}^{M}\widehat{f_m}) = \frac{1}{M^2}\sum_{m=1}^{M}\text{Var}(\widehat{f_m}) = \frac{1}{M}\text{Mean}(\text{Var}(\widehat{f_1}, \widehat{f_2}, ..., \widehat{f_M}))$$

- Random forests introduce 2 levels of randomness to encourage independence:
  1. Bootstrapping sample (randomly draw a subset of samples for each decision tree)
  2. For each tree split, randomly select a subset of features
- Random forests uses the bagging principle to use the ensemble to reduce variance

Boosted decision trees: Boosting

- For boosting, we use a different approach
- Analogous to algorithms like gradient descent where for each iteration, we move closer to minimizer
- Consider estimate at iteration t, $\widehat{f_t}$, then

$$\widehat{f_{t+1}} = \widehat{f_t} + \alpha_t h_t$$

  where α_t is step-size and h_t is a single decision tree
- At each iteration a decision tree is chosen to try to fit the residuals $Y - \widehat{f_t}(X)$
- So each step reduces bias but increases variance by adding a another decision tree and need to decide stopping criteria T
- Final ensemble estimator:

$$\widehat{f} = \widehat{f_T} = \widehat{f_0} + \sum_{t=0}^{T-1} \alpha_t h_t$$

Feature selection through MDI

- We know how to calculate MDI for each feature of a single decision tree (based on Titanic example)
- Now for an ensemble of trees

$$\widehat{f} = \sum_{m=1}^{M} \alpha_m \widehat{f_m}$$

  simply use the co-efficients α_m to get a weighted MDI for each feature within each tree
- This is an embedded method because all the MDI's have already been computed during the computation of the decision tree ensemble

MDI challenges and limitations

- Typically works well when there is categorical features and responses with a small number of categories
- Captures non-linear and interaction effects that linear models don't
- However tend to bias towards features that have a large number of categories/are continuous because they tend to appear more

Motivating synthetic model

Assume $p = 5$ but only $s = 2$ relevant features:

$$Y = 1(X_1 > 0) \oplus 1(X_2 > 0) + \epsilon$$

where X_1, X_2 independent Normal$(0, 1)$, $\epsilon \sim$ Bernoulli$(1/2)$, Y is binary

- $\oplus$ refers to XOR operation (one or the other but not both)
- Naturally arises in many settings (e.g. Epistatis in biology)

Motivating synthetic model code

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegressionCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

np.random.seed(0)

n = 500
X = np.random.randn(n, 5)

y = ((X[:, 0] > 0) ^ (X[:, 1] > 0)).astype(int)

X = pd.DataFrame(X, columns=[f"x{j}" for j in range(X.shape[1])])
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0
)
```

Lasso for synthetic example

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

lasso = LogisticRegressionCV(
    penalty="l1",
    solver="saga",
    cv=5,
    max_iter=5000,
    random_state=0
)

lasso.fit(X_train_scaled, y_train)

lasso_coef = pd.Series(lasso.coef_[0], index=X.columns)
lasso_selected = lasso_coef[lasso_coef != 0]

lasso_selected
```

```
x0    0.072470
x2    0.054703
x3    0.155139
x4    0.181657
dtype: float64
```

MDI for synthetic example

```
rf = RandomForestClassifier(
    n_estimators=300,
    min_samples_leaf=10,
    random_state=0,
    n_jobs=-1
)

rf.fit(X_train, y_train)

mdi = pd.Series(rf.feature_importances_, index=X.columns)
mdi.sort_values(ascending=False)
```

```
x1    0.337488
x0    0.306470
x4    0.132633
x3    0.123652
x2    0.099757
dtype: float64
```

```
accuracy_score(y_test, rf.predict(X_test))
```

```
0.9133333333333333
```

```
accuracy_score(y_test, lasso.predict(X_test_scaled))
```

```
0.56
```

MDI and random forest for California housing data

```python
from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=500,
    min_samples_leaf=10,
    random_state=0,
    n_jobs=-1
)

rf.fit(X_train, y_train)

mdi = pd.Series(rf.feature_importances_, index=X.columns)
mdi.sort_values(ascending=False
```

```
median_income          0.583438
longitude              0.155513
latitude               0.140024
housing_median_age     0.061072
population             0.021760
total_bedrooms         0.018647
total_rooms            0.011402
households             0.008144
dtype: float64
```

Top 5 features according to MDI

```
k = 5
mdi_topk = mdi.sort_values(ascending=False).head(k).index
mdi_topk
```

```
Index(['median_income', 'longitude', 'latitude', 'housing_median_age',
       'population'],
      dtype='object')
```

Comparison of top 5 features MDI and Lasso

- 4 of the 5 features the same for Logistic Lasso and MDI
- One difference is ‘Population‘ for MDI and ‘Number of Bedrooms‘ for Lasso
- Population may only be relevant in some locations (’Latitude’ and ’Longitude’)
- Population may have splits near the extremes
- Population interacts with ’Income’ and ’Occupancy’
- ’Number of Bedrooms’ more like a standard linear relationship
- All of these conjectures can be verified by looking at the data